

Dashboard interactivo

Violencia de género e intrafamiliar

Alcaldía de Bucaramanga · datos SIVIGILA

Ciencia de datos con Python, Pandas y Streamlit

1. Equipo y roles

Integrante	Rol principal	Participación
Frank Edwin Tabares Gil	Datos / Pandas	Limpieza, filtros, groupby , agregaciones
Daniel Esteban Lozano	Interfaz Streamlit	Navegación, secciones, estilos
Kevin David Galeano Tabares	Interfaz Streamlit	Navegación, secciones, estilos
Juan Diego Gómez Higueta	Despliegue y pruebas	GitHub, Streamlit Cloud, validación

Todos participaron en revisiones, pruebas y documentación.

2. Propósito del proyecto

Desarrollar una **aplicación analítica modular** sobre datos reales de **SIVIGILA** (Bucaramanga) que:

- Separe **ingesta de datos**, **reglas de transformación** y **interfaz de usuario** en módulos distintos
- Demuestre **buenas prácticas de arquitectura** (código mantenible y trabajo en equipo)
- Aplique **Pandas** para exploración, limpieza, filtrado y agregación
- Ofrezca **visualización interactiva** (Plotly) y **acceso web** vía Streamlit en local y en la **nube**

3. Objetivos generales

1. **Diseñar una arquitectura por capas:** paquete `dashboard` (constantes, estilos, datos, transformaciones, vistas)
2. **Centralizar configuración** (nombres de columnas, opciones de Plotly) en un solo módulo reutilizable
3. **Ingerir** el CSV con encoding robusto y ruta relativa al proyecto
4. **Explorar** el dataset (`head` , `describe` , `value_counts`)
5. **Garantizar calidad:** estrategias documentadas con `fillna` / `dropna` en columnas clave
6. **Transformar y resumir:** filtros, `sort_values` , `groupby` + `agg`
7. **Comunicar** con gráficos y **filtros globales** (comuna, año, área, tipo, sexo)
8. **Desplegar** el repositorio en **Streamlit Community Cloud**

4. ¿De qué trata el dataset? (contexto)

- Registros de **violencia de género e intrafamiliar** notificados en el marco de **vigilancia en salud pública** en **Bucaramanga** (Santander).
- Cada fila representa un **caso o evento** asociado a atención o notificación en el sistema **SIVIGILA**, con datos sociodemográficos, territoriales y clínicos-administrativos.
- El conjunto permite **explorar patrones** por tiempo (año, mes), **zona** (comuna, barrio), **tipo de violencia**, **sexo y edad** de la víctima, entre otros.
- **Importancia:** apoya el **análisis descriptivo** y la toma de decisiones en salud pública; en el curso lo usamos con fines **académicos** de manipulación de datos.

5. Origen y sistema: SIVIGILA

- **SIVIGILA** (Sistema de Vigilancia en Salud Pública) es el sistema oficial en Colombia para **notificar y consolidar** eventos de interés en salud pública.
- Los datos provienen de la **Alcaldía de Bucaramanga** (y contexto departamental/municipal según el archivo).
- **Formato:** archivo **CSV** con **miles de registros** y muchas columnas **categorías** (texto) y **temporales**.
- En la app tratamos el CSV como **DataFrame de Pandas**: carga, revisión de nulos, filtros y agregaciones para gráficos.

6. Variables relevantes en el CSV

Ámbito	Ejemplos de columnas
Hecho	Tipo de violencia (<code>def_naturaleza</code>), fecha, hora, escenario
Víctima	Sexo, grupo de edad, ciclo de vida
Territorio	Comuna, barrio, área (<code>area_</code>)
Tiempo	Año, mes, semana epidemiológica
Otros	Parentesco con agresor, tipo de seguridad social, UPGD (<code>nom_upgd</code>)

No todas se usan en cada gráfico; las elegimos según el objetivo de cada sección.

7. Demo — estructura de la app

Menú lateral — cinco vistas (`dashboard/views/`):

1. **Inicio** — propósito, objetivos, métricas y técnicas de curso
2. **Explorar datos** — tablas y frecuencias
3. **Limpieza y transformación** — nulos, filtros, orden, reporte por año
4. **Análisis estadístico** — tablas resumen
5. **Visualizaciones** — gráficos con **filtros que actualizan toda la vista**

Demo en vivo: una comuna + rango de años + gráficos interactivos (zoom, hover).

8. Pandas — conceptos de clase

Tema	Uso en el proyecto
Filtrado	<code>isin</code> , <code>&</code> , condiciones por columna
Sondeo	<code>unique</code> , <code>nunique</code> , <code>value_counts</code>
Orden	<code>sort_values</code> (ej. por año descendente)
Limpieza	<code>fillna("Sin información")</code> , <code>dropna(subset=...)</code>
Agrupación	<code>groupby</code> + <code>agg</code> (<code>count</code> , <code>nunique</code>)

9. Arquitectura del código (modular)

app.py	← Solo entrada: config, sidebar, carga datos, enruta vistas
dashboard/	
├── constants.py	← Nombres de columnas, PLOTLY_CONFIG
├── styles.py	← CSS y pie de página
├── sidebar.py	← Navegación lateral
├── data.py	← cargar_datos() + @st.cache_data
├── transforms.py	← preparar_df_visual, filtrar_para_graficos
└── views/	
├── inicio.py	
├── explorar.py	
├── limpieza.py	
├── analisis.py	
└── visualizaciones.py	← Plotly (bar, pie, line, imshow)

- **Entrada:** `streamlit run app.py`
- **Dependencias:** `requirements.txt` · entorno: `venv`

10. Por qué esta arquitectura

- `app.py` **delgado**: fácil de leer; cada pantalla vive en su archivo.
- **Separación de responsabilidades**: datos $\neq$ transformación $\neq$ UI.
- **Mantenimiento**: cambiar columnas del CSV se hace sobre todo en `constants.py`.
- **Colaboración**: distintas personas pueden trabajar en `views/` sin conflictos masivos.
- Streamlit sigue el modelo **re-ejecutar el script**; `@st.cache_data` en `data.py` evita releer el CSV en cada clic.

11. Entorno virtual (`venv`) y despliegue

- `venv` : carpeta aislada con una instalación de Python y paquetes **solo para este proyecto** (evita conflictos con otros proyectos o con el Python global).
- `pip install -r requirements.txt` : instala las **mismas versiones** de librerías en cualquier máquina o servidor.
- **Git + GitHub**: historial de cambios y **código + CSV** disponibles en la nube.
- **Streamlit Community Cloud**: conecta el repo, ejecuta `streamlit run app.py` y publica una **URL** `*.streamlit.app`.

12. Python — ¿qué es y qué hace aquí?

- **Qué es:** lenguaje de programación **interpretado**, muy usado en **ciencia de datos**, automatización y web.
- **Qué hace en el proyecto:** es el **lenguaje base** donde corren **Pandas** (manipulación de tablas), **Streamlit** (interfaz) y **Plotly** (gráficos).
- **Ventaja:** una sola base de código para **todo el flujo** desde leer el CSV hasta mostrar gráficos en el navegador.

13. Pandas — ¿qué es y qué hace aquí?

- **Qué es:** librería de Python para **datos tabulares**; su estructura central es el **DataFrame** (tabla con filas y columnas con nombre).
- **Qué hace aquí:** lee el **CSV**, detecta tipos, **filtra** filas, **rellena o elimina** nulos, **ordena**, **agrupa** y **cuenta** para alimentar tablas y gráficos.
- **Por qué la usamos:** es el **estándar** en el curso y en la industria para limpieza y análisis exploratorio sobre datos reales.

14. Streamlit — ¿qué es y qué hace aquí?

- **Qué es:** framework de Python para crear **aplicaciones web de datos** con pocas líneas: tablas, gráficos, formularios, texto.
- **Qué hace aquí:** define la **interfaz** (barra lateral, secciones), muestra **métricas**, **DataFrames** y incrusta gráficos de Plotly con `st.plotly_chart`.
- **Por qué la usamos:** desarrollo **rápido** para proyectos académicos y prototipos sin montar un backend tradicional aparte.

15. Plotly — ¿qué es y qué hace aquí?

- **Qué es:** librería de **visualización interactiva** (zoom, pan, información al pasar el mouse, exportar imagen).
- **Qué hace aquí:** genera **barras, tortas, líneas y mapas de calor** a partir de datos ya agregados o filtrados con Pandas (`plotly.express`).
- **Diferencia útil:** frente a gráficos estáticos, el usuario **explora** el gráfico en la propia web sin regenerar código.

16. Git, GitHub y Streamlit Cloud

Herramienta	Qué es	Qué hace en el proyecto
Git	Sistema de control de versiones	Registra cambios en archivos (<code>commit</code> , <code>push</code>)
GitHub	Plataforma que aloja el repositorio	Backup, colaboración y enlace con Streamlit Cloud
Streamlit Cloud	Servicio de hosting para apps Streamlit	Despliega <code>app.py</code> desde GitHub y da URL pública

17. Conclusiones

- Se cubrió el flujo **datos** → **limpieza** → **análisis** → **visualización** → **nube**
- Los **filtros por zona y tiempo** orientan el análisis territorial y temporal
- **fillna** vs **dropna** muestra cómo la estrategia de nulos altera el tamaño y la interpretación del dataset

18. Logros · dificultades · aprendizajes

Logros: aplicación funcional, despliegue web, práctica integral de Pandas

Dificultades: nulos en variables clave, encoding del CSV, coherencia de filtros y gráficos

Aprendizajes: calidad y documentación de datos son base de conclusiones válidas

Gracias

Preguntas